Контест 1 · Python-разминка, Git и спеки

Неделя 2 · темы недель 1–2: Git-процесс, `AGENTS.md`, user story и критерии приёмки, протокол API, бот и FastAPI. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

> Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 14 обычных, 3 со звёздочкой, 3 частей босса — всего **58** монет. Задача засчитывается, только если вы можете объяснить свой код.

A. Строки и условия

A1. Из имени ветки достаньте номер issue. Если номера нет или ветка не по правилам `тип/номер-slug`, верните `None`.

```
Ввод:  issue_number("feature/12-board")
Вывод: 12

Ввод:  issue_number("main")
Вывод: None
```

A2. Дан список сообщений коммитов. Напечатайте только те, что относятся к области `game` (начинаются с `game:`). Коммиты слияния (начинаются с `Merge`) пропускаем. Дошли до строки `--` — заканчиваем.

```
Ввод:  ["game: поле 6×7", "Merge pull request #3", "bot: /start", "game: ход", "--", "game:
сюда не дойдём"]
Вывод: game: поле 6×7
       game: ход
```

A3. Ревьюеры по очереди оставляют вердикты `"approve"` или `"changes"`. PR можно мержить, если есть хотя бы одно `"approve"` и после последнего `"changes"` кто-то ещё раз поставил `"approve"`.

```
Ввод:  can_merge(["changes", "approve"])
Вывод: True

Ввод:  can_merge(["approve", "changes"])
Вывод: False
```

B. Словари и множества

B1. Дан список авторов коммитов (по одному на коммит). Верните список пар `(автор, число коммитов)`, по убыванию числа коммитов, при равенстве — по алфавиту.

```
Ввод:  top_authors(["Маша", "Петя", "Маша", "Аня", "Петя", "Маша"])
Вывод: [("Маша", 3), ("Петя", 2), ("Аня", 1)]
```

B2. Роли учеников лежат в словаре: имя → (основная, дополнительная). Верните множество ролей из списка `ROLES`, которые никто не взял **основной**.

```
ROLES = {"аналитика", "бэкенд", "фронтенд", "тестирование", "devops", "дизайн"}

Ввод:  free_roles({"Аня": ("бэкенд", "devops"), "Петя": ("фронтенд", "дизайн"), "Маша":
("бэкенд", "аналитика")})
Вывод: {"аналитика", "тестирование", "devops", "дизайн"}
```

B3. Даны списки файлов, которые меняют два открытых PR. Верните отсортированный список файлов, на которых возможен конфликт. Без вложенных циклов.

```
Ввод:  conflict_files(["bot/main.py", "api/rooms.py", "README.md"], ["api/rooms.py",
"web/app.js", "README.md"])
Вывод: ["README.md", "api/rooms.py"]
```

B4*. Восемь учеников надо разделить на две подкоманды по четыре так, чтобы суммарный опыт (число решённых задач за прошлый год) отличался как можно меньше. Верните минимальную разницу. Подсказка: `itertools.combinations`.

```
Ввод:  split_teams([10, 20, 30, 40, 50, 60, 70, 80])
Вывод: 0        # 10+40+50+80 = 20+30+60+70 = 180
```

C. Спеки и протоколы

C1. В тексте issue критерии приёмки — строки-чекбоксы `- [ ]` и `- [x]`. Верните процент выполненных, округлённый до целого. Если чекбоксов нет — `None`.

```
Ввод:  done_percent("Задача\n- [x] падает вниз\n- [ ] полный столбец\n- [x] тесты")
Вывод: 67
```

C2. User story пишется по шаблону «Как <роль>, я хочу <действие>, чтобы <ценность>». Разберите строку в словарь, а если она не по шаблону — верните `None`.

```
Ввод:  parse_story("Как игрок, я хочу пригласить друга, чтобы сыграть вдвоём")
Вывод: {"role": "игрок", "want": "пригласить друга", "why": "сыграть вдвоём"}
```

C3. По протоколу игры клиент присылает ход `{"type": "move", "col": <0..6>}`. Напишите `validate_move(msg)`, которая возвращает список ошибок (пустой — всё хорошо). Проверьте: это словарь, `type` равен `"move"`, `col` — целое число (не `bool`!) от 0 до 6, лишних ключей нет.

```
Ввод:  validate_move({"type": "move", "col": 3})
Вывод: []

Ввод:  validate_move({"type": "move", "col": "3", "cheat": 1})
Вывод: ["col должен быть целым числом", "лишние ключи: cheat"]
```

D. Git

D1. Заведите репозиторий под контест и коммитьте каждую решённую задачу отдельно, по правилу `область: что сделано` (область — буква раздела: `a: ...`, `b: ...`). Покажите `git log --oneline`, где хотя бы три коммита.

```
Ввод:  git log --oneline
Вывод: a3f9c21 c: процент выполненных критериев
       7be4d02 b: топ авторов коммитов
       1c0e5aa a: номер issue из имени ветки
```

D2. В учебном монорепо создайте issue с критериями приёмки, ветку `feature/<номер>-...`, сделайте изменение и откройте PR по шаблону (разделы «Что сделано», «Как проверить», «Что сгенерировал агент», строка `Closes #N`). Получите одобрение и влейте.

D3*. Устройте конфликт: в двух ветках по-разному поменяйте одну строку в `AGENTS.md` и слейте обе в `main`. Разрешите конфликт так, чтобы в файле остались оба правила. Покажите `git log --oneline --graph`.

E. FastAPI и бот

E1. `GET /ping` отвечает `{"pong": true}`. `GET /rooms/{room_id}` отдаёт комнату из словаря `ROOMS`, а если её нет — статус 404.

```
Ввод:  GET /rooms/abc        # ROOMS = {"abc": {"players": ["Аня"]}}
Вывод: {"id": "abc", "players": ["Аня"]}

Ввод:  GET /rooms/zzz
Вывод: 404 {"detail": "Комната не найдена"}
```

E2. Бот на aiogram 3: на `/start` отвечает приветствием и кнопкой «Играть», которая открывает Mini App (`WebAppInfo`, адрес — любой ваш HTTPS-адрес). Покажите в Telegram. Токен — только из переменной окружения.

E3. Сгенерируйте агентом `POST /rooms` (создаёт комнату с уникальным `id`) и проведите ревью его кода: найдите и запишите хотя бы два замечания (проверка входа, ответ 201, коды ошибок, лишний код). Сдаёте код и список замечаний.

E4 · босс. Доска задач команды на FastAPI. За каждую часть свой плюс. Колонки: `backlog`, `todo`, `in_progress`, `review`, `done`.

а) `POST /issues` с телом `{"title": "..."}` создаёт задачу в `backlog` (ответ 201, у задачи `id`, `title`, `status`). `GET /issues` отдаёт все задачи, а с параметром `?status=review` — только задачи из этой колонки.

б) `POST /issues/{id}/move` с телом `{"to": "todo"}` двигает задачу. Разрешены только переходы `backlog → todo → in_progress → review → done` и возврат `review → in_progress`. Запрещённый переход — 409, нет задачи — 404.

в) `GET /board` возвращает колонки со списками `id` и процент задач в `done` (округлить до целого, на пустой доске 0).

```
Ввод:  POST /issues {"title": "Поле 6×7"}
Вывод: 201 {"id": 1, "title": "Поле 6×7", "status": "backlog"}

Ввод:  POST /issues/1/move {"to": "done"}
Вывод: 409 {"detail": "Нельзя перейти из backlog в done"}

Ввод:  GET /board
Вывод: {"backlog": [1], "todo": [], "in_progress": [], "review": [], "done": [],
"done_percent": 0}
```

E5*. Тесты на E4 через `TestClient`: хотя бы по одному на каждую часть и один на запрещённый переход.

```
Ввод:  pytest
Вывод: 4 passed
```